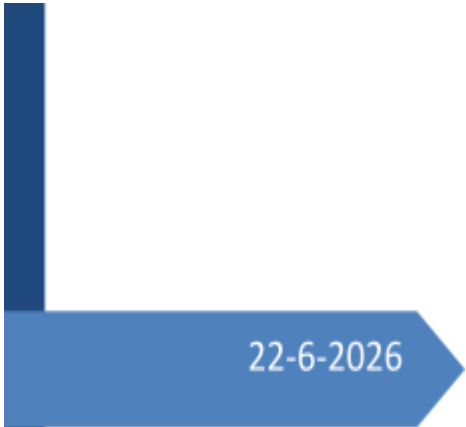
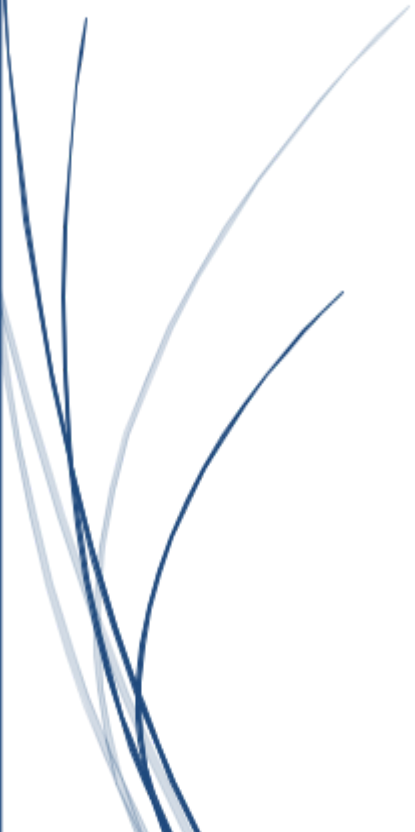




22-6-2026

TPI PROGRAMACION 2

Tecnicatura Superior en Programación
Sistema de Gestión Food Store (Backend
Corporativo)



Integrantes:

ACOSTA CRISTINA
GALLARDO LUCAS
MESAS RAMIRO
TORRES SHIRLEY

ÍNDICE

Entregables Técnicos del Repositorio	Página 3
Marco Teórico y Tecnológico	Página 3
Decisiones Técnicas y Arquitectura del Sistema	Página 4
Detalle y Responsabilidad de Paquetes	Página 4
Dificultades Técnicas y Soluciones Aplicadas	Página 5
Enlaces del Proyecto y Webgrafía	Página 6

1. ENTREGABLES TÉCNICOS (REPOSITORIO GITHUB)

El desarrollo del sistema está respaldado por los siguientes componentes técnicos cargados de forma estructurada en la plataforma GitHub:

Proyecto Java Completo

Código fuente estructurado bajo el paradigma de programación orientada a objetos, modularizado de manera estricta mediante paquetes jerárquicos independientes.

Script SQL (Schema)

Archivo estructurado de definición de datos para la generación del esquema relacional pedidos_db, incluyendo restricciones de claves primarias, foráneas y tablas base para el módulo de categorías.

Configuración de Persistencia

Implementación de conectividad JDBC encapsulada en la clase de control de conexiones para interactuar de forma directa con el motor MySQL sin requerir herramientas externas de ORM.

Archivo README.md

Documento técnico ubicado en la raíz del repositorio que provee la guía paso a paso de instalación de la base de datos, la parametrización de variables de entorno y los comandos necesarios para la compilación y ejecución del proyecto.

2. MARCO TEÓRICO Y TECNOLÓGICO

La arquitectura del software se diseñó implementando tecnologías consolidadas dentro del ecosistema de desarrollo de sistemas empresariales y aplicando patrones orientados a la separación de responsabilidades:

Tecnologías Utilizadas

Java Standard Edition (Java SE): Lenguaje base robusto y fuertemente tipado que permite la implementación del paradigma de programación orientada a objetos, herencia, encapsulamiento y el manejo estructurado de excepciones en tiempo de ejecución.

MySQL Engine: Sistema de gestión de bases de datos relacionales utilizado para garantizar la persistencia del estado del negocio, la atomicidad de las operaciones y la integridad de los datos a través de restricciones lógicas.

Java Database Connectivity (JDBC): API nativa de Java encargada de gestionar el puente de comunicación, la traducción de comandos relacionales y el procesamiento seguro de transacciones hacia la base de datos.

Conceptos Clave Aplicados

Arquitectura Multicapa: Patrón arquitectónico enfocado en desasociar la interfaz de interacción, la lógica de negocio y los mecanismos de acceso a datos, impidiendo el acoplamiento rígido de código y facilitando el mantenimiento modular.

Data Access Object (DAO): Patrón de diseño que centraliza y aísla las operaciones de lectura y escritura en la base de datos, proveyendo una interfaz limpia a las capas de lógica superior.

Baja Lógica (Soft Delete): Técnica de persistencia donde los registros no son destruidos físicamente mediante instrucciones DELETE, sino modificados a través de una bandera booleana (eliminado = true), resguardando el historial transaccional y la integridad referencial.

3. DECISIONES TÉCNICAS Y ARQUITECTURA

Para lograr un software escalable, se adoptó una arquitectura en capas distribuidas horizontalmente. La interacción se inicia en el menú de la consola interactiva, el cual delega el control al servicio de negocio; este valida las reglas instituidas y solicita los servicios del DAO, el cual finalmente interactúa de forma directa con la base de datos utilizando la conexión provista por la infraestructura técnica.

Flujo Conceptual de Datos

[Interfaz de Consola: Main] → [Negocio: Service] → [Persistencia: DAO] → [Infraestructura: ConexionDB] → [Motor Relacional: MySQL]

```
=====
Intentando conectar a pedidos_db...
=====
❖EXITO TOTAL! Tu código de NetBeans se conectó perfectamente a MySQL.
=====

=== FOOD STORE - MENU PRINCIPAL ===
1. GESTION DE CATEGORIAS
2. GESTION DE PRODUCTOS
3. GESTION DE USUARIOS
4. GESTION DE PEDIDOS
0. SALIR
Seleccione una opcion:
```

4. DETALLE Y RESPONSABILIDAD DE PAQUETES

El código fuente se segmentó rigurosamente en los siguientes paquetes estructurales para asegurar la independencia de componentes:

integrado.prog2.config

Contiene la infraestructura de conectividad a través de la clase ConexionDB. Implementa un bloque estático para registrar de forma explícita el driver nativo de comunicación (com.mysql.cj.jdbc.Driver) y expone el método estático getConnection() encargado de retornar objetos de tipo Connection parametrizados con la URL de persistencia local.

integrado.prog2.entities

Agrupar las clases de modelado del dominio de datos aplicando principios avanzados de herencia:

- Base (Clase Abstracta): Define la estructura troncal heredable compartida por las entidades de negocio. Define los atributos obligatorios id (clave primaria) y eliminado (bandera de borrado lógico).
- Categoria (Clase Concreta): Hereda directamente de la clase base abstracta y extiende el comportamiento incorporando los campos de negocio especializados nombre y descripcion.
- Calculable (Interfaz): Establece el contrato abstracto mediante el método nominal calcularTotal(), fijando el estándar operativo para módulos financieros o transaccionales futuros.

integrado.prog2.enums

Estandariza los dominios lógicos fijos del negocio para evitar inconsistencias operativas por tipeo manual:

- Estado: Valores tipados para el control del ciclo de vida del pedido (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO).
- Rol: Restricciones de seguridad del perfil del usuario del sistema (ADMIN, USUARIO).
- FormaPago: Medios transaccionales admitidos por la plataforma (TARJETA, TRANSFERENCIA, EFECTIVO).

integrado.prog2.exception

Implementa una jerarquía desacoplada de manejo de fallos para la capa de software. Define excepciones no verificadas especializadas como DataAccessException, CategoriaException y CategoriaNotFoundException que extienden de RuntimeException, permitiendo capturar de forma semántica anomalías sin forzar firmas engorrosas en los métodos superiores.

integrado.prog2.dao

Contiene la clase CategoriaDAO. Implementa los bloques de consulta SQL y comandos preparados. Emplea la técnica destructiva controlada de try-with-resources para garantizar el cierre seguro de canales abiertos de memoria y mapea los resultados relacionales provenientes de tablas físicas hacia colecciones estructuradas orientadas a objetos.

- **ProductoDAO:** Implementa bloques de consulta SQL para la creación de los productos, y el manejo de los datos persistidos a la BD, como también un manejo de eliminado lógico y algunos métodos útiles como mapearProducto que nos sirve para el mapeo sin redundancia de código o la búsqueda por id de un producto.
- **UsuarioDAO:** Implementa bloques de consulta SQL para la creación de los usuarios, también manejando los datos de usuario persistidos en la BD, con un método útil llamado existeEmail para comprobar si un email ya existe y no guardar un usuario con el mismo email existente.

integrado.prog2.menuus

Contiene la clase MenuProducto, para mostrar las opciones que tenemos del producto y orquestar el uso del servicio.

integrado.prog2.service

Representa el núcleo operativo del negocio mediante la clase FoodStoreService: Valida restricciones críticas (por ejemplo, consistencia del nombre, no duplicidad de categorías activas) antes de permitir transferencias a la persistencia y arroja las excepciones personalizadas del sistema si detecta quiebres en las reglas operativas.

- **ProductoService:** Métodos que manejan la lógica de negocio sobre la creación, edición, eliminación y listado de los productos. Junto con eso también la validación del stock, precio, y una categoría existente.
- **UsuarioService:** Métodos que manejan la lógica de negocio sobre la creación, edición, eliminación y listado de los usuarios. Junto con eso también la validación del email y verificación de que no exista ya en la BD.

integrado.prog2.main (Clase Main)

Despliega la consola interactiva mediante bucles condicionales estructurados. Posee la responsabilidad de gestionar las excepciones semánticas que burbujan desde los servicios inferiores, abstrayendo el error técnico crudo y traduciéndolo en notificaciones potentes y legibles para el usuario sin desestabilizar la ejecución de la aplicación.

5. DIFICULTADES TÉCNICAS Y SOLUCIONES APLICADAS

Obstáculo Técnico Detectado	Impacto en la Solución	Resolución e Ingeniería Aplicada
Fuga de Recursos de Memoria (Resource Leaks)	Saturación de conexiones activas en el motor de bases de datos MySQL al mantener abiertos canales de instrucciones de forma indefinida.	Se implementó el patrón estructural try-with-resources en el DAO, forzando la destrucción automatizada de objetos Connection y

Obstáculo Técnico Detectado	Impacto en la Solución	Resolución e Ingeniería Aplicada
		PreparedStatement al concluir la instrucción.
Acoplamiento Técnico Crudo (Exposición de SQLException)	La capa de presentación del Main dependía de librerías nativas java.sql, violando el principio de independencia arquitectónica.	Se diseñó el paquete de excepciones personalizadas. El DAO encapsula las SQLException nativas mediante bloques catch y las relanza transmutadas en DataAccessException semánticas.
Quiebre de Integridad Referencial por Borrado Físico	El uso de instrucciones destructivas DELETE eliminaba registros históricos de la base de datos rompiendo auditorías o restricciones de claves.	Se implementó el mecanismo de Borrado Lógico (Soft Delete) heredado de la clase base abstracta, mutando la propiedad de persistencia a través de actualizaciones de banderas lógicas (eliminado = true).
Inestabilidad ante Entrada de Datos Erróneos	El ingreso accidental de valores alfanuméricos en menús numéricos de consola provocaba detenciones abruptas del hilo principal de ejecución (Crash).	Se incorporó un bloque de control de errores generalizado en la clase Main aislando excepciones del tipo NumberFormatException y reseteando los flags de navegación de manera segura.

6. ENLACES DEL PROYECTO Y WEBGRAFÍA

Enlaces Oficiales del Proyecto Final

Repositorio de Código Fuente (GitHub, rama mainFinal):

https://github.com/lucas-dev-cl/TPI_Programaci-n_UTN

Video Explicativo de Arquitectura y Defensa Oral:

<https://youtu.be/A7jn0fDUfAE>

Bibliografía y Referencias de Apoyo Técnico

- Oracle Corporation. (2026). Java Standard Edition Development Kit Documentation (JDK 21). Recuperado de la documentación oficial de Oracle.
- MySQL Enterprise. (2026). MySQL Connector/J Developer Guide and Reference Manual. Recuperado del portal oficial de desarrolladores de MySQL.

- Bloch, J. (2018). Effective Java (3rd Edition). Addison-Wesley Professional. Aplicado para los patrones de diseño orientados a objetos y gestión limpia de excepciones.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. Utilizado para la estructuración, legibilidad y modularización de la arquitectura en capas del sistema.

22 de junio de 2026